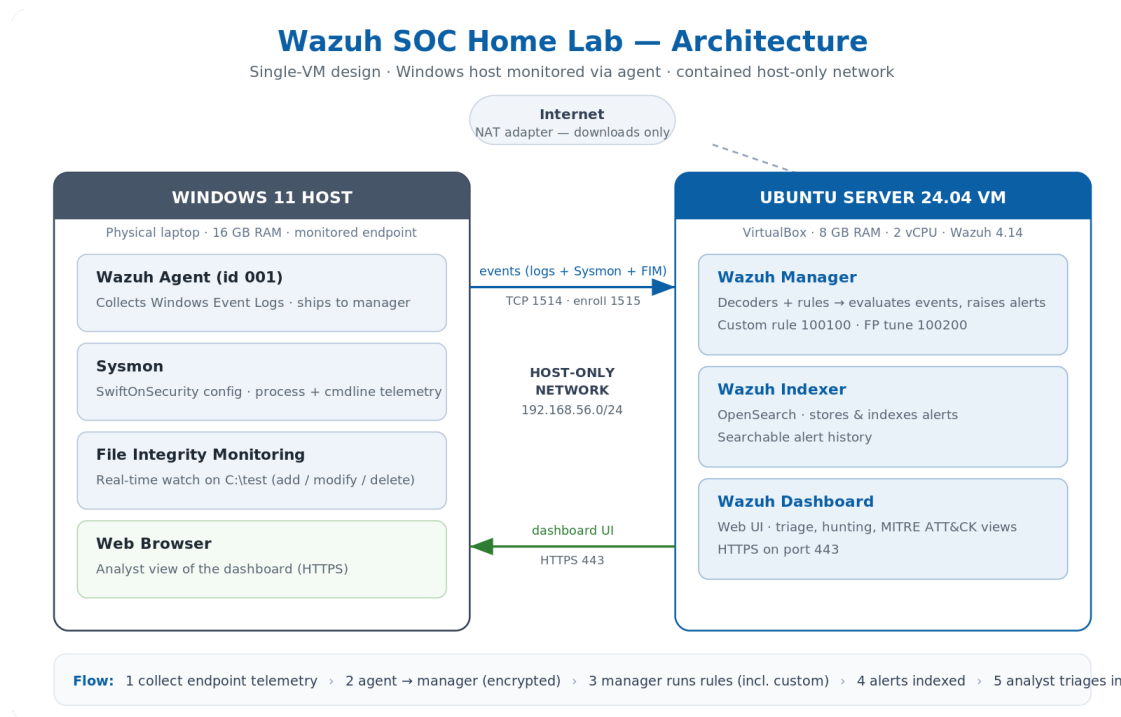


Wazuh SOC Home Lab: SIEM, Detection Engineering & Alert Triage

A self-hosted Security Operations Center (SOC) home lab built on Wazuh. I stood up a full SIEM, onboarded a real Windows endpoint with Sysmon and File Integrity Monitoring, simulated attacks, triaged the alerts, wrote a custom detection rule that catches obfuscation the built-in rule misses, and tuned a critical-severity false positive. I documented the failures and the fixes as I went, because that part is most of the learning.

Author: Sanjish KC · **Repo:** github.com/sanjishkc/wazuh-soc-home-lab

Architecture



Architecture diagram

I used a single-VM design to fit constrained hardware. Only the Wazuh server runs in a VM, and the Windows host itself is the monitored endpoint via a lightweight agent. The two talk over an isolated host-only network, so no lab traffic leaves the machine. NAT is there only so the VM can download packages.

Table of Contents

1. Overview
2. Skills Demonstrated

3. [Environment & Tooling](#)
 4. [Build Walkthrough](#)
 5. [Troubleshooting Story: The Dashboard That Wouldn't Install](#)
 6. [File Integrity Monitoring](#)
 7. [Detection 1: New Local Admin Account \(T1136\)](#)
 8. [Detection 2: Encoded PowerShell \(T1059.001\)](#)
 9. [Detection Engineering: Custom Rule \(#6\)](#)
 10. [Detection Engineering: False-Positive Tuning \(#7\)](#)
 11. [Lessons Learned](#)
 12. [Repository Structure](#)
 13. [Reproduce It](#)
 14. [References](#)
-

Overview

This project builds a working SOC and then does the day job of a SOC analyst: generate security telemetry, detect simulated attacks, map each detection to the MITRE ATT&CK framework, and read, write, and tune detection rules.

I built it on modest hardware (an 8 GB laptop, later upgraded to 16 GB). I did not hide that constraint. It drove a single-VM architecture and, at one point, a disk-space failure that I had to diagnose from logs and fix.

What the project shows an employer:

- I can deploy and run a SIEM (manager, indexer, and dashboard) from scratch.
 - I can onboard endpoints and enrich their telemetry with Windows Event Logs, Sysmon, and FIM.
 - I can triage an alert down to who did what to whom, and decode an obfuscated payload.
 - I can do detection engineering: read an existing rule, find its blind spot, and write a better one.
 - I can tune noise without going blind to the real threat.
 - I write things down, including the parts that broke.
-

Skills Demonstrated

Area	Evidence in this project
SIEM deployment & operations	Wazuh all-in-one install, service management, post-reboot recovery
Endpoint onboarding	Wazuh agent on Windows, agent enrollment over host-only network
Telemetry enrichment	Sysmon (SwiftOnSecurity config), File

Alert triage	Integrity Monitoring Read alerts to subject/target/SID; escalation judgment by severity
Threat detection	New admin account (T1136), encoded PowerShell (T1059.001)
Detection engineering	Custom rule 100100 that catches cases built-in 92057 misses
Alert tuning	Surgical false-positive suppression of critical rule 92213
Payload analysis	Decoded base64 -EncodedCommand to reveal intent
MITRE ATT&CK mapping	Techniques tagged on every detection
Linux / infrastructure troubleshooting	LVM disk expansion, broken dpkg recovery, service auth-sync

Environment & Tooling

Item	Selection	Notes
Hypervisor	Oracle VirtualBox 7.2	Snapshots for safe rollbacks
Guest OS	Ubuntu Server 24.04.4 LTS	Server (no GUI); supported by Wazuh 4.14
SIEM	Wazuh 4.14.6 (assisted all- in-one)	Manager, indexer, and dashboard
Endpoint	Windows 11 host (16 GB RAM)	Monitored via Wazuh agent (id 001)
Endpoint telemetry	Sysmon + SwiftOnSecurity config	Process, command-line, file, and registry visibility
Server VM	8 GB RAM, 2 vCPU, 54 GB disk	Sized for the all-in-one stack
Networking	NAT + Host-only (192.168.56.0/24)	Internet for downloads; contained host-to-VM link

Build Walkthrough

1. Ubuntu Server VM

I created an Ubuntu Server 24.04 VM in VirtualBox with two adapters: NAT for downloads and Host-only for the contained host-to-VM link. One VirtualBox 7.2 gotcha caught me here. The second network adapter only appears once you switch the Network settings to Expert mode.

```
Installing system [ Help ]

subiquity/Ad/apply_autoinstall_config:
subiquity/Late/apply_autoinstall_config:
configuring apt
curtin command in-target
installing system
executing curtin install initial step
executing curtin install partitioning step
curtin command install
configuring storage
  running 'curtin block-meta simple'
  curtin command block-meta
  removing previous storage devices
  configuring disk: disk-sda
  configuring partition: partition-0
  configuring partition: partition-1
  configuring format: format-0
  configuring partition: partition-2
  configuring lvm_voigroup: lvm_voigroup-0
  configuring lvm_partitions: lvm_partition-0
  configuring format: format-1
  configuring mount: mount-1
  configuring mount: mount-0
executing curtin install extract step
curtin command install
writing install sources to disk
  running 'curtin extract'
  curtin command extract
  acquiring and extracting image from cp:///tmp/tmpulbrvf2/mount
configuring keyboard
curtin command in-target
executing curtin install curthooks step
curtin command install
configuring installed system
  running 'curtin curthooks'
  curtin command curthooks
  configuring apt configuring apt
  installing missing packages
  Installing packages on target system: ['grub-pc']
  configuring iscsi service
  configuring raid (mdadm) service
  configuring NVMe over TCP
  installing kernel /

[ View full log ]
```

```
Ubuntu 24.04.4 LTS wazuh-sever tty1

wazuh-sever login: [ 42.751722] cloud-init[895]: Generating locales (this might take a while)...
[ 46.397979] cloud-init[895]: en_US.UTF-8... done
[ 46.406672] cloud-init[895]: Generation complete.
[ 48.824158] cloud-init[1082]: Cloud-init v. 25.2-0ubuntu1~24.04.1 running 'modules:final' at Sat, 18 Jul 2026 22:19:21 +0000. Up 48.67 seconds.
ci-info: no authorized SSH keys fingerprints found for user snzs.
[14]Jul 18 22:19:22 cloud-init: #####
[14]Jul 18 22:19:22 cloud-init: -----BEGIN SSH HOST KEY FINGERPRINTS-----
[14]Jul 18 22:19:22 cloud-init: 256 SHA256:vcSVUUnw/h6uwo9SJ2InB8EqZQF8RAtdofQEe2o50U root@wazuh-sever (ECDSA)
[14]Jul 18 22:19:22 cloud-init: 256 SHA256:zIE5S0v8J08WYz60aQlK5J+t7GSaCfXG6nTQwJvGul root@wazuh-sever (ED25519)
[14]Jul 18 22:19:22 cloud-init: 3872 SHA256:2fSo10gHKSFJaw6QgHJxPvH2kaTp2VzSr8mpU0RZJQ root@wazuh-sever (RSA)
[14]Jul 18 22:19:22 cloud-init: -----END SSH HOST KEY FINGERPRINTS-----
[14]Jul 18 22:19:22 cloud-init: #####
-----BEGIN SSH HOST KEY KEYS-----
ecdsa-sha2-nistp256 AAAAE2VjZHNhLlNoTiItbWlzdHAuNTYAAAAIbmlzdzhhYjNTYAAABBBPMzYhNr8VaP8ktXoojehkEz1F+yDQaQlWIZG6EFgmX9S8lveyard5TnPV30yBme1xzpq0GZu2GRZJu/Pd8PDCow=
root@wazuh-sever
ssh-ed25519 AAAAC3NzaC1lZD01INTESAAAAIAxRRt3bUEK5xb0T3D0XJL+1s56R76vRjy0Xxt4Tr5 root@wazuh-sever
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCAQCyv45Dp06uwoNoEqJhR2RxcEINiv4w5u1gUHVITUL09R18RWa2SL1oX6Jzxws0418LSR7gQ0ySmSID0npwtr2k7xmDHRUAYERbolZ225UVTvdH9u3vmA5
E1XH9n6Lah1xmMUI6r26GK5S2pEUL1fHfSV0CCNS27Lb0emQVvYmgetouXyVGH3EXM3d/xuhueae0Req1UpHR2Luz1CLeytQqHNJ0Uvp52Y12YJM+vk1DJOktJN/Q+ausonxTzeeV2X1yG4kRFQqWmLunkJf0K
9goY2NGuR002R9v/C9y5amCFKsrT1YvKHrREfSPvAk5f7FrV2m65MMdd8mTs7eccpX05VLJzNkTV3HfM70pp4SYB6x6lwsml+gSwlckTDxk96msSp2YR1IQkukgtRHmTbIdA2aBkY9HX9mNEJTJGF7oh7pufU7
90ldTP1aU4ea120JokIuv+/v8HmMxIF1NMAUhl5o2S2Yk1grE1ldRLQ+cc1Hox57kyWME= root@wazuh-sever
-----END SSH HOST KEY KEYS-----
[ 49.398322] cloud-init[1082]: Cloud-init v. 25.2-0ubuntu1~24.04.1 finished at Sat, 18 Jul 2026 22:19:22 +0000. DataSource DataSourceNone. Up 49.37 seconds
```

2. Low-RAM safety net: swap

Before installing Wazuh I added a swap file so a memory spike would not trigger the out-of-memory killer.

```
snzs@wazuh-sever:~$ sudo swapon /swapfile
snzs@wazuh-sever:~$ echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
/swapfile none swap sw 0 0
snzs@wazuh-sever:~$ free -h
Command 'free' not found, did you mean:
  command 'free' from deb procps (2:4.0.4-4ubuntu3.2)
Try: sudo apt install <deb name>
snzs@wazuh-sever:~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	3.8Gi	421Mi	3.4Gi	1.1Mi	215Mi	3.4Gi
Swap:	6.8Gi	0B	6.8Gi			

```
snzs@wazuh-sever:~$ _
```

Swap configured before Wazuh install

3. Wazuh install (assisted, all-in-one)

`curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a`

```
snzs@wazuh-sever:~$ curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a

19/07/2026 00:52:14 INFO: Starting Wazuh installation assistant. Wazuh version: 4.14.6
19/07/2026 00:52:15 INFO: Verbose logging redirected to /var/log/wazuh-install.log
19/07/2026 00:52:39 INFO: Verifying that your system meets the recommended minimum hardware requirements.
19/07/2026 00:52:39 INFO: Wazuh web interface port will be 443.
19/07/2026 00:52:49 INFO: --- Dependencies ---
19/07/2026 00:52:49 INFO: Installing apt-transport-https.
19/07/2026 00:53:01 INFO: Installing debhelper.
19/07/2026 00:54:04 INFO: Wazuh repository added.
19/07/2026 00:54:04 INFO: --- Configuration files ---
19/07/2026 00:54:04 INFO: Generating configuration files.
19/07/2026 00:54:06 INFO: Generating the root certificate.
19/07/2026 00:54:06 INFO: Generating admin certificates.
19/07/2026 00:54:07 INFO: Generating Wazuh indexer certificates.
19/07/2026 00:54:08 INFO: Generating Filebeat certificates.
19/07/2026 00:54:08 INFO: Generating Wazuh dashboard certificates.
19/07/2026 00:54:09 INFO: Created wazuh-install-files.tar. It contains the Wazuh cluster key, certificates, and passwords necessary for installation.
19/07/2026 00:54:10 INFO: --- Wazuh indexer ---
19/07/2026 00:54:10 INFO: Starting Wazuh indexer installation.
19/07/2026 00:56:34 INFO: Wazuh indexer installation finished.
19/07/2026 00:56:34 INFO: Wazuh indexer post-install configuration finished.
19/07/2026 00:56:34 INFO: Starting service wazuh-indexer.
19/07/2026 00:57:11 INFO: wazuh-indexer service started.
19/07/2026 00:57:11 INFO: Initializing Wazuh indexer cluster security settings.
19/07/2026 00:57:22 INFO: Wazuh indexer cluster security configuration initialized.
19/07/2026 00:57:22 INFO: Wazuh indexer cluster initialized.
19/07/2026 00:57:22 INFO: --- Wazuh server ---
19/07/2026 00:57:22 INFO: Starting the Wazuh manager installation.
19/07/2026 01:04:24 INFO: Wazuh manager installation finished.
19/07/2026 01:04:25 INFO: Wazuh manager vulnerability detection configuration finished.
19/07/2026 01:04:25 INFO: Starting service wazuh-manager.
19/07/2026 01:04:53 INFO: wazuh-manager service started.
19/07/2026 01:04:53 INFO: Starting Filebeat installation.
19/07/2026 01:07:47 INFO: Filebeat installation finished.
19/07/2026 01:08:49 INFO: Filebeat post-install configuration finished.
19/07/2026 01:08:49 INFO: Starting service filebeat.
19/07/2026 01:08:53 INFO: filebeat service started.
19/07/2026 01:08:53 INFO: --- Wazuh dashboard ---
19/07/2026 01:08:53 INFO: Starting Wazuh dashboard installation.
```

Wazuh installing

The indexer, manager, and Filebeat installed cleanly. The dashboard step failed repeatedly, which is its own story below. Once the root cause was fixed, the install completed:

```

23/07/2026 12:03:59 INFO: wazuh-manager service started.
23/07/2026 12:03:59 INFO: Starting Filebeat installation.
23/07/2026 12:05:12 INFO: Filebeat installation finished.
23/07/2026 12:05:20 INFO: Filebeat post-install configuration finished.
23/07/2026 12:05:20 INFO: Filebeat post-install configuration finished.
23/07/2026 12:05:20 INFO: Starting service filebeat.
23/07/2026 12:05:25 INFO: filebeat service started.
23/07/2026 12:05:25 INFO: --- Wazuh dashboard ---
23/07/2026 12:05:25 INFO: Starting Wazuh dashboard installation.
23/07/2026 12:18:44 INFO: Wazuh dashboard installation finished.
23/07/2026 12:18:44 INFO: Wazuh dashboard post-install configuration finished.
23/07/2026 12:18:45 INFO: Starting service wazuh-dashboard.
23/07/2026 12:18:46 INFO: wazuh-dashboard service started.
23/07/2026 12:18:49 INFO: Updating the internal users.
23/07/2026 12:19:11 INFO: A backup of the internal users has been saved in the /etc/wazuh-indexer/internalusers-backup folder.
23/07/2026 12:19:48 INFO: The filebeat.yml file has been updated to use the Filebeat Keystore username and password.
23/07/2026 12:20:54 INFO: Initializing Wazuh dashboard web application.
23/07/2026 12:20:55 INFO: Wazuh dashboard web application not yet initialized. Waiting...
23/07/2026 12:21:14 INFO: Wazuh dashboard web application not yet initialized. Waiting...
23/07/2026 12:21:30 INFO: Wazuh dashboard web application initialized.
23/07/2026 12:21:30 INFO: --- Summary ---
23/07/2026 12:21:30 INFO: You can access the web interface https://<wazuh-dashboard-ip>:443
User: admin
Password: [REDACTED]
23/07/2026 12:21:30 INFO: Installation finished.
snzs@wazuh-server:~$

```

Wazuh install finished successfully

The dashboard is reachable over the host-only network at https://192.168.56.102. Note that it is HTTPS on port 443, not HTTP.

The screenshot shows the Wazuh dashboard interface. The top navigation bar includes the Wazuh logo, a menu icon, and the 'Overview' tab. The main content area is divided into several sections:

- AGENTS SUMMARY:** A message stating 'This instance has no agents registered. Please deploy agents to begin monitoring your endpoints.' with a 'Deploy new agent' button.
- LAST 24 HOURS ALERTS:** A summary of alerts by severity: Critical (0), High (0), Medium (194), and Low (200).
- ENDPOINT SECURITY:** Includes 'Configuration Assessment', 'Malware Detection', and 'File Integrity Monitoring'.
- THREAT INTELLIGENCE:** Includes 'Threat Hunting', 'Vulnerability Detection', and 'MITRE ATT&CK'.

Below these sections, there is a table of agents. The first agent is 'Snzs-PC' with ID 001, status 'active', IP address 192.168.56.1, and version Wazuh v4.14.6. The table also shows system inventory details for the agent, including 4 cores, 15.9GB memory, and Intel(R) Core(TM) i7-6500U CPU @ 2.50GHz.

At the bottom, there are charts for 'Events count evolution', 'MITRE ATT&CK Top Tactics' (showing Defense Evasion), and 'Compliance' (showing PCI DSS).

4. Onboard the Windows endpoint and Sysmon

I deployed the Wazuh agent to the Windows host, pointed at the manager's host-only IP. Then I installed Sysmon with the SwiftOnSecurity config and added a <localfile> block so Wazuh ingests the Microsoft-Windows-Sysmon/Operational channel. Sysmon's command-line visibility is what makes the later detections useful, since it records the exact command that ran.

Troubleshooting Story: The Dashboard That Wouldn't Install

The Wazuh dashboard failed to install four times, rolling the whole stack back each time. This is the part of the project I am most glad I documented, because it shows how I work a problem instead of guessing.

```
19/07/2026 00:52:39 INFO: Verifying that your system meets the recommended minimum hardware requirements.
19/07/2026 00:52:49 INFO: --- Dependencies ---
19/07/2026 00:52:49 INFO: Installing apt-transport-https.
19/07/2026 00:53:01 INFO: Installing debhelper.
19/07/2026 00:54:04 INFO: Wazuh repository added.
19/07/2026 00:54:04 INFO: --- Configuration files ---
19/07/2026 00:54:04 INFO: Generating configuration files.
19/07/2026 00:54:06 INFO: Generating the root certificate.
19/07/2026 00:54:06 INFO: Generating Admin certificates.
19/07/2026 00:54:07 INFO: Generating Wazuh indexer certificates.
19/07/2026 00:54:08 INFO: Generating Filebeat certificates.
19/07/2026 00:54:08 INFO: Generating Wazuh dashboard certificates.
19/07/2026 00:54:09 INFO: Created wazuh-install-files.tar. It contains the Wazuh cluster key, certificates, and passwords necessary for installation.
19/07/2026 00:54:10 INFO: --- Wazuh indexer ---
19/07/2026 00:54:10 INFO: Starting Wazuh indexer installation.
19/07/2026 00:56:34 INFO: Wazuh indexer installation finished.
19/07/2026 00:56:34 INFO: Wazuh indexer post-install configuration finished.
19/07/2026 00:56:34 INFO: Starting service wazuh-indexer.
19/07/2026 00:57:11 INFO: wazuh-indexer service started.
19/07/2026 00:57:11 INFO: Initializing Wazuh indexer cluster security settings.
19/07/2026 00:57:22 INFO: Wazuh indexer cluster security configuration initialized.
19/07/2026 00:57:22 INFO: Wazuh indexer cluster initialized.
19/07/2026 00:57:22 INFO: --- Wazuh server ---
19/07/2026 00:57:22 INFO: Starting the Wazuh manager installation.
19/07/2026 01:04:24 INFO: Wazuh manager installation finished.
19/07/2026 01:04:25 INFO: Wazuh manager vulnerability detection configuration finished.
19/07/2026 01:04:25 INFO: Starting service wazuh-manager.
19/07/2026 01:04:53 INFO: wazuh-manager service started.
19/07/2026 01:04:53 INFO: Starting Filebeat installation.
19/07/2026 01:07:47 INFO: Filebeat installation finished.
19/07/2026 01:08:49 INFO: Filebeat post-install configuration finished.
19/07/2026 01:08:49 INFO: Starting service filebeat.
19/07/2026 01:08:53 INFO: filebeat service started.
19/07/2026 01:08:53 INFO: --- Wazuh dashboard ---
19/07/2026 01:08:53 INFO: Starting Wazuh dashboard installation.
19/07/2026 01:21:19 ERROR: Wazuh dashboard installation failed.
19/07/2026 01:21:19 INFO: --- Removing existing Wazuh installation ---
19/07/2026 01:21:19 INFO: Removing Wazuh manager.
19/07/2026 01:21:24 INFO: Wazuh manager removed.
19/07/2026 01:21:29 INFO: Removing Wazuh indexer.
19/07/2026 01:21:40 INFO: Wazuh indexer removed.
19/07/2026 01:21:40 INFO: Removing Filebeat.
19/07/2026 01:21:53 INFO: Filebeat removed.
19/07/2026 01:21:53 INFO: Removing Wazuh dashboard.
19/07/2026 01:22:06 INFO: Wazuh dashboard removed.
19/07/2026 01:22:07 INFO: Installation cleaned. Check the /var/log/wazuh-install.log file to learn more about the issue.
snzs@wazuh-server:~$
```

Dashboard installation failed

My first theory was memory. I assumed the dashboard's memory-heavy build step was getting OOM-killed, so I added swap, raised the VM from 4 GB to 5 GB, and eventually upgraded the laptop to 16 GB and gave the VM 8 GB. It still failed at the same step. That ruled memory out.

```
snzs@wazuh-server:~$
snzs@wazuh-server:~$
snzs@wazuh-server:~$ curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a
[sudo] password for snzs:
19/07/2026 01:38:49 INFO: Starting Wazuh installation assistant. Wazuh version: 4.14.6
19/07/2026 01:38:49 INFO: Verbose logging redirected to /var/log/wazuh-install.log
19/07/2026 01:38:52 ERROR: Wazuh manager already installed.
19/07/2026 01:38:52 INFO: If you want to overwrite the current installation, run this script adding the option -o/--overwrite. This will erase all the existing
configuration and data.
snzs@wazuh-server:~$
```

Install error recurring

So I stopped guessing and read the log:

```
sudo grep -iE "error|fail|no space|killed" /var/log/wazuh-install.log | tail -40
```

The real error was sitting right there:

E: Write error - write (28: No space left on device)

```
Jul 19 01:47:31 wazuh-sever systemd[1]: Starting wazuh-manager.service - Wazuh manager...
Jul 19 01:47:31 wazuh-sever env[3777]: /usr/bin/env: '/var/ossec/bin/wazuh-control': No such file or directory
Jul 19 01:47:31 wazuh-sever systemd[1]: wazuh-manager.service: Control process exited, code=exited, status=127/n/a
Jul 19 01:47:31 wazuh-sever systemd[1]: wazuh-manager.service: Failed with result 'exit-code'.
Jul 19 01:47:31 wazuh-sever systemd[1]: Failed to start wazuh-manager.service - Wazuh manager.
19/07/2026 01:47:31 INFO: --- Removing existing Wazuh installation ---
19/07/2026 01:47:39 INFO: Removing Wazuh manager.
Reading package lists...
Building dependency tree...
Reading state information...
The following packages were automatically installed and are no longer required:
  libfwupd2 libgusb2
Use 'sudo apt autoremove' to remove them.
The following packages will be REMOVED:
  wazuh-manager*
0 upgraded, 0 newly installed, 1 to remove and 0 not upgraded.
After this operation, 1,114 MB disk space will be freed.
(Reading database ... 117171 files and directories currently installed.)
Removing wazuh-manager (4.14.6-1) ...
dpkg: error processing package wazuh-manager (--remove):
  installed wazuh-manager package pre-removal script subprocess returned error exit status 127
dpkg: too many errors, stopping
Errors were encountered while processing:
 wazuh-manager
Processing was halted because there were too many errors.
E: Sub-process /usr/bin/dpkg returned an error code (1)
19/07/2026 01:47:39 WARNING: The Wazuh manager package could not be removed.
19/07/2026 01:47:39 INFO: Removing Wazuh Indexer.
Reading package lists...
Building dependency tree...
Reading state information...
The following packages were automatically installed and are no longer required:
  libfwupd2 libgusb2
Use 'sudo apt autoremove' to remove them.
The following packages will be REMOVED:
  wazuh-indexer*
0 upgraded, 0 newly installed, 1 to remove and 0 not upgraded.
After this operation, 1,103 MB disk space will be freed.
(Reading database ... 117171 files and directories currently installed.)
Removing wazuh-indexer (4.14.6-1) ...
Running Wazuh Indexer Pre-Removal Script
Stop existing wazuh-indexer.service
(Reading database ... 116009 files and directories currently installed.)
Purging configuration files for wazuh-indexer (4.14.6-1) ...
dpkg: warning: while removing wazuh-indexer, directory '/var/log/wazuh-indexer' not empty so not removed
dpkg: warning: while removing wazuh-indexer, directory '/var/lib/wazuh-indexer' not empty so not removed
dpkg: warning: while removing wazuh-indexer, directory '/etc/wazuh-indexer' not empty so not removed
19/07/2026 01:47:44 INFO: Wazuh Indexer removed.
19/07/2026 01:47:44 INFO: Installation cleaned.
snzs@wazuh-sever:~$
```

No space left on device

The root cause was the Ubuntu LVM “half the disk” default. The guided install had given the root filesystem only about 27 GB of the 50 GB disk and left the rest unused in the volume group:

```
df -h /      # root only 27G, filling during install
sudo vgs     # VSize 54G, VFree 27G <-- half the disk unallocated
```

The fix was two commands. Grow the logical volume, then grow the filesystem inside it:

```
sudo lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv # grow the logical volume
sudo resize2fs /dev/ubuntu-vg/ubuntu-lv           # grow the ext4 filesystem
df -h /      # now 54G, 43G free
```

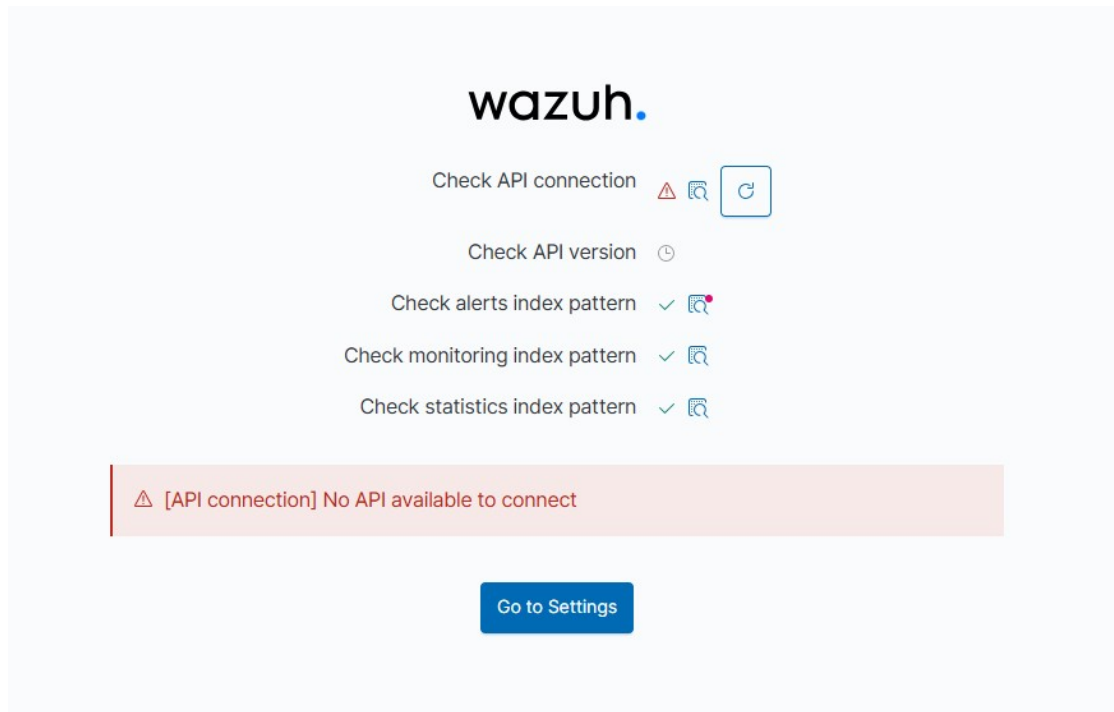
There was a second obstacle. The failed installs had left a half-removed wazuh-manager package whose uninstall script errored, plus orphaned processes still holding ports 1515 and 55000. I cleaned it by hand:

```
sudo pkill -9 -f wazuh
sudo rm -f /var/lib/dpkg/info/wazuh-manager.prerm /var/lib/dpkg/info/wazuh-manager.postrm
```

```
sudo dpkg --purge --force-all wazuh-manager
sudo rm -rf /var/ossec
```

A clean re-run then succeeded. The lesson I took from this: read the log before you touch anything. I spent time (and money on RAM) on a memory theory when the real problem was disk space from Ubuntu's default LVM layout.

One more issue showed up after a reboot. The dashboard reported [API connection] No API available to connect and then an auth-token error. Restarting only the manager had left its API and the indexer's security out of sync.



API connection error

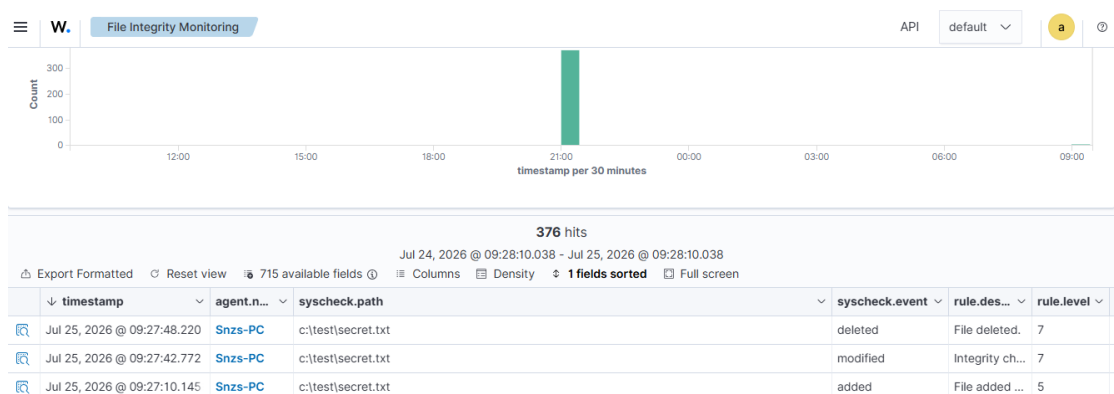
The fix was to restart the whole stack in order:

```
sudo systemctl restart wazuh-indexer    # (wait ~60s)
sudo systemctl restart wazuh-manager    # (wait ~30s)
sudo systemctl restart wazuh-dashboard
```

File Integrity Monitoring

I added a real-time FIM watch on C:\test in the agent config, then created, modified, and deleted a file. All three events fired with file hashes and a content diff.

```
<directories check_all="yes" realtime="yes" report_changes="yes">C:
\test</directories>
```



FIM: file added, modified, deleted

Event

File added

File modified

File deleted

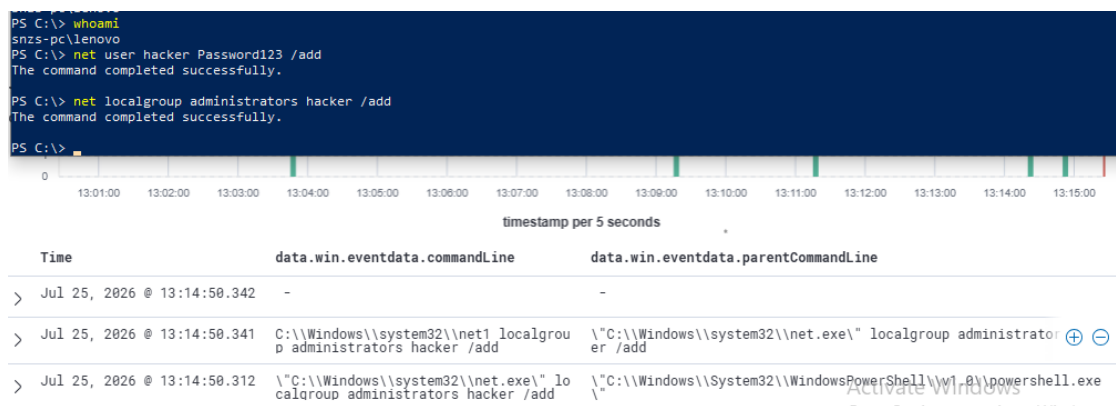
A tuning note worth recording: Wazuh's `<ignore>` is global, with no per-directory option. A default sregex ignore for `.log/.htm/.jpg/...` was silently suppressing `.log` files in the monitored folder. Good reminder that “monitored” does not mean “reported” when an ignore rule matches the path.

Detection 1: New Local Admin Account (T1136)

The simulation, run on the host and cleaned up afterward:

```
net user hacker Passw0rd123! /add
net localgroup administrators hacker /add
net user hacker /delete           # cleanup
```

That one action produced a cluster of correlated alerts across two data sources: Windows Security events and Sysmon process-create events showing the literal command line.



User creation alert caught

The alert to escalate is rule.id 60154, "Administrators Group Changed", at level 12, from Windows Event 4732. Reading it down to who did what to whom:

- Subject (who): LENOVO, the acting account
- Target (what): the Administrators group, targetSid S-1-5-32-544, which is the built-in local Administrators SID on every Windows machine
- firedtimes: 2, because it caught both the add (4732) and the cleanup removal (4733)

timestamp per 5 seconds					
rule.description	rule.level	data.win.system.eventID	rule.mitre.id	rule.mitre.technique	data.win.eventdata.commandLine
Administrators Group Changed	12	4732	T1484	Domain Policy Modification	-
Discovery activity executed	3	1	T1087	Account Discovery	C:\Windows\system32\net1 localgroup administrators hacker /add
Discovery activity spawned via powershell execution	3	1	T1087, T1059.001	Account Discovery, PowerShell	"C:\Windows\system32\net.exe" localgroup administrators hacker /add
Users Group Changed	5	4732	T1484	Domain Policy Modification	-
User account changed	8	4738	T1098	Account Manipulation	-
User account enabled or created	8	4722	T1098	Account Manipulation	-

Detection 1: rule ID, MITRE, event ID detail

One detection-engineering observation. Wazuh tagged this as T1484 (Domain Policy Modification), which is imprecise for a local admin-group change. T1098 or T1136 fit it better. The stock rule-to-ATT&CK mappings are useful but not always exact.

Detection 2: Encoded PowerShell (T1059.001)

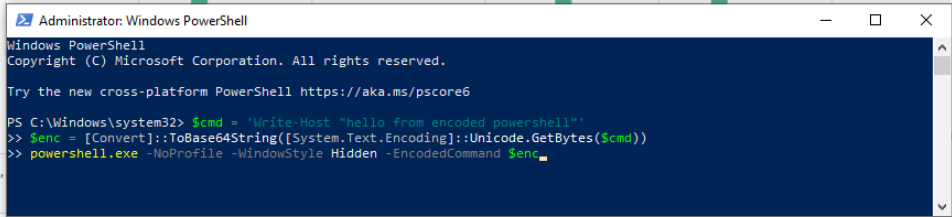
The simulation uses a harmless payload wrapped in the obfuscation shape attackers actually use (-NoProfile -WindowStyle Hidden -EncodedCommand):

```
$cmd = 'Write-Host "hello from encoded powershell"'
$enc = [Convert]::ToBase64String([System.Text.Encoding]::Unicode.GetBytes($cmd))
powershell.exe -NoProfile -WindowStyle Hidden -EncodedCommand $enc
```

Jul 25, 2026 @ 13:43:44.589 12	Powershell.exe spawned a powershell process which executed a base64 encoded command	PowerShell	\C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -NoProfile -WindowStyle Hidden -EncodedCommand VwByAGkAdABlAC0ASABvAHMAdAqACIAaABlAGwAbABvACAAZgByAG8AbQAgAGUAbgBjAG8AZABlAGQAIABwAG8AdwBIAHIAcwBoAGUAbABsACIA
--------------------------------	---	------------	--

Encoded PowerShell command

Wazuh caught it out of the box at level 12, rule.id 92057, "Powershell.exe spawned a powershell process which executed a base64 encoded command."

Time		13:45:00
Jul 25,		andLine
Jul 25, 2026 @ 13:43:44.589 12	Powershell.exe spawned a powershell process which executed a base64 encoded command	PowerShell
		\C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -NoProfile -WindowStyle Hidden -EncodedCommand VwByAGkAdABlAC0ASABvAHMAdAqACIAaABlAGwAbABvACAAZgByAG8AbQAgAGUAbgBjAG8AZABlAGQAIABwAG8AdwBIAHIAcwBoAGUAbABsACIA

Encoded PowerShell alert in Wazuh

Then the analyst move: decode the payload. An attacker's -EncodedCommand hides what they actually ran, so I decoded it to reveal the intent:

[System.Text.Encoding]::Unicode.GetString([Convert]::FromBase64String('<base64>'))

PS C:\Windows\system32> [System.Text.Encoding]::Unicode.GetString([Convert]::FromBase64String('VwByAGkAdABlAC0ASABvAHMAdAqACIAaABlAGwAbABvACAAZgByAG8AbQAgAGUAbgBjAG8AZABlAGQAIABwAG8AdwBIAHIAcwBoAGUAbABsACIA'))
Write-Host "hello from encoded powershell"
PS C:\Windows\system32>

Decoded payload found in the log

In a real incident this same step reveals the malware's true command, whether that is a download URL, a follow-on payload, or something else.

Detection Engineering: Custom Rule (#6)

I read the built-in rule that caught Detection 2 (92057) and found two blind spots:

1. It requires parentImage = powershell.exe, so it misses encoded PowerShell launched from cmd.exe, an Office macro, or wscript.exe.
2. Its regex lists en|enco|encode|encodedcommand but skips enc and encoded, which are the abbreviations attackers most often use.

My custom rule (rules/local_rules.xml, id 100100, level 12) drops the parent requirement and closes the abbreviation gap:

```

<rule id="100100" level="12">
  <if_group>sysmon_event1 </if_group>
  <field name="win.eventdata.commandLine" type="pcre2">(?!powershell\.exe.+
  \-\\b(encodedcommand|encoded|encode|enco|enc|en|ec|ea|e))\b</field>
  <description>Encoded PowerShell executed (any parent process, incl. -enc/-
  encoded) - possible obfuscation [custom]</description>
  <mitre><id>T1059.001 </id></mitre>
</rule>

```

Encoded PowerShell executed (any parent process) - possible obfuscation [custom]

[View alerts of this rule](#)

Information

ID	Level	File	Path
100100	12	local_rules.xml	etc/rules

Groups

local, windows, sysmon

Details

If_group	Win.Eventdata.CommandLine
sysmon_event1	pattern: (?i)powershell\.exe.+\\-\\b(encodedcommand e ea ec encodeda encode en enco)\\b, type: pcre2

Compliance

Activate Windows

Custom rule 100100 in local_rules.xml

To prove it fills the gap, I launched powershell.exe -nop -w hidden -enc <base64> from cmd.exe, which is exactly the case stock rule 92057 misses (wrong parent, plus the -enc abbreviation). Rule 100100 fired at level 12 where 92057 stayed silent.

```

> Jul 25, 2026 @ 14:30:23.751 100100 Encoded PowerShell executed (any parent process) - possible obfuscation [custom] powershell.exe -nop -w hidden -enc VwByAGkAdABlAC0ASABvAHMAdAAgACIAaABlAGwABvACAAZgByAG8AbQAgAGUA5gBjAG8AZABlAGQAIABwAG8AdwBIAHIACwBoAGUAbABSACTA

```

Custom rule 100100 fired

How I would describe it in an interview:

I analysed Wazuh's built-in encoded-PowerShell rule, found it only fires when PowerShell is the parent and that it missed the -enc and -encoded abbreviations, and I wrote a custom rule that detects encoded PowerShell regardless of the parent process.

Detection Engineering: False-Positive Tuning (#7)

Ranking alerts by volume and severity turned up something worth chasing. Level 15 (critical) alerts were firing around 25 times a day. Critical alerts should be rare, so I looked into it.

They were rule.id 92213, “Executable file dropped in folder commonly used by malware” (T1105), firing on files like:

C:\Users\...\AppData\Local\Temp_PSScriptPolicyTest_<random>.<random>.ps1

The random-looking, ever-changing filenames almost looked like DNS-style exfiltration, so I investigated to a conclusion rather than assuming. It turned out to be PowerShell’s own execution-policy self-test: a throwaway .ps1 written to Temp, run to check the policy, then deleted. Local file creation by the signed system powershell.exe, no network, and random temp naming. A benign behaviour caught by rule 92213’s broad “any script in Temp” heuristic, which made it a false positive at the highest severity.

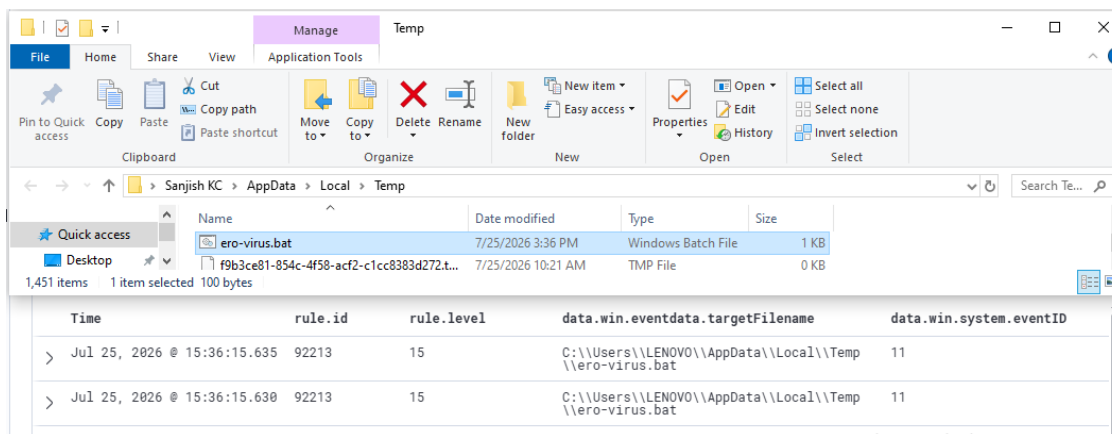
The tune (rules/local_rules.xml, id 100200, level 0) silences only the self-test and leaves 92213 live for any real drop:

```
<rule id="100200" level="0">
  <if_sid>92213</if_sid>
  <field name="win.eventdata.targetFilename" type="pcre2">(?  
i) _PSScriptPolicyTest_ </field>
  <description>Benign PowerShell execution-policy self-test in Temp - tuned FP,  
rule 92213 stays live for real drops [custom]</description>
</rule>
```

```
</rule>
<rule id="100200" level="0">
  <if_sid>92213</if_sid>
  <field name="win.eventdata.targetFilename" type="pcre2">(?!)_PSScriptPolicyTest_</field>
  <description>Benign PowerShell execution-policy self-test in Temp - tuned FP, rule 92213 stays live for real drops [custom]</description>
</rule>
```

Local rule added to tune the false positive

I checked both directions. After the tune, the PowerShell self-tests went silent, but dropping a real ero-virus.bat into Temp still fired 92213 at level 15. The noise is gone and the signal is intact.



False positive silenced, real drop still detected

A false positive at level 15 does more damage than a hundred at level 3, because analysts start ignoring the critical queue. That is why I tuned it with a specific condition instead of muting the whole rule.

Lessons Learned

1. Fit the architecture to the hardware. A single-VM design let a full SIEM run on a constrained laptop.
 2. Match the OS to the tool. I used Ubuntu 24.04, not the newest 26.04, because Wazuh 4.14 supports it.
 3. Read the log before acting. The dashboard failures were disk space, not RAM.
 4. Ubuntu's guided LVM install often uses only half the disk. lvextend plus resize2fs reclaims it.
 5. To fix a broken dpkg package, blank its prerm and postrm scripts, then force-purge.
 6. After an API auth-token error, restart the stack in order: indexer, then manager, then dashboard.
 7. Sysmon's commandLine field is what turns "a process ran" into a real detection.
 8. Read the built-in rule before you write or tune. Its logic shows you the exact gap.
 9. Tune false positives with a specific condition (if_sid plus a field match plus level 0) rather than muting a rule that also catches real threats.
 10. A critical false positive is the most damaging kind, because it trains analysts to ignore the top-priority queue.
-

Repository Structure

```
wazuh-soc-home-lab/
├── README.md           # this document
├── images/
│   ├── architecture.svg / .png # architecture diagram
│   └── screenshots/         # evidence for every step
```

```
| rules/
|   | local_rules.xml      # custom rule 100100 + FP tune 100200
| docs/
|   | Wazuh-SOC-Home-Lab-Report.pdf # printable version of this document
|   | engineering-log.md    # detailed log: issue, diagnosis, fix
```

Reproduce It

1. Create an Ubuntu Server 24.04 VM with NAT and Host-only adapters. Expand the LVM root to use the full disk.
 2. `curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a`
 3. Deploy the Wazuh agent to a Windows host, pointed at the manager's host-only IP.
 4. Install Sysmon with the SwiftOnSecurity config and add the Microsoft-Windows-Sysmon/Operational localfile block to the agent.
 5. Drop `rules/local_rules.xml` into `/var/ossec/etc/rules/` and restart the manager.
 6. Run the Detection 1 and Detection 2 simulations and confirm the alerts and the custom rule fire.
-

References

- [Wazuh documentation](#) and [installation guide](#)
 - [Sysmon \(Sysinternals\)](#) and [SwiftOnSecurity sysmon-config](#)
 - [MITRE ATT&CK](#): T1136, T1059.001, T1105, T1098
 - [Wazuh ruleset reference](#)
-

Built and documented by Sanjish KC as a hands-on blue-team / SOC portfolio project. Every screenshot is from my own lab.